

Universidad del Valle de Guatemala

Facultad de Ingeniería

CC3067 Redes, sección 10

Laboratorio 5

WordPress en Arquitectura de Dos Capas

Nicolás Concuá, 23197

Daniela Ramírez, 23053

Fernando Mendoza, 19644

Guatemala, 10 de noviembre de 2026

Índice

1. Introducción 3
2. Entorno y versiones 3
3. Topología y direccionamiento 4
4. Implementación 5
    4.1 Anfitrión y máquinas virtuales 5
    4.2 Acceso por SSH 5
    4.3 Firewall 6
    4.4 Base de datos 6
    4.5 Servidor web y WordPress 7
    4.6 Permisos 7
    4.7 SFTP y FileZilla 8
    4.8 HTTPS 10
    4.9 Resolución local 12
5. Cliente y servidor 12
6. Pruebas 13
7. Problemas encontrados 14
8. Conclusiones 14
Referencias 15
Anexo. Reconstrucción del laboratorio 15

1. Introducción

En este laboratorio montamos un sitio WordPress en dos máquinas virtuales: una para el servidor web y otra para la base de datos. Las dos corren Ubuntu Server 26.04.1 LTS sobre QEMU/KVM y se administran solo por SSH. El sitio se publica por HTTPS con un certificado de Let's Encrypt y se abre desde el cliente con el nombre `redes-lab5-wp.duckdns.org`, que se resuelve con el archivo hosts.

La configuración está en scripts dentro del repositorio del grupo (github.com/nicoCT04/Lab5-Redes). Todos leen los mismos parámetros de `config/lab5.env`, y las contraseñas están en `secretos.env`, que no se sube al repositorio. Con esos scripts el laboratorio se puede reconstruir desde cero siguiendo los pasos de la sección 4.

2. Entorno y versiones

Las máquinas virtuales corren en QEMU 8.2.2 con aceleración KVM. KVM es el módulo del kernel de Linux que ejecuta las VM directamente en el procesador usando VT-x, y QEMU crea cada VM como un proceso con discos y tarjetas de red *virtio*. Lo elegimos porque es software libre, se maneja por completo desde la línea de comandos y es la misma base que usan Proxmox y OpenStack.

Para instalar el sistema usamos la imagen cloud oficial de Ubuntu Server con cloud-init en lugar del instalador. Cada VM queda lista en alrededor de un minuto, con el nombre, el usuario, la llave SSH y la red definidos en archivos de texto.

Tabla 1. Versiones utilizadas.

Componente	Versión
Anfitrión	Ubuntu 24.04 LTS, QEMU 8.2.2 con KVM
Máquinas virtuales	Ubuntu Server 26.04.1 LTS, 2 vCPU, 2 GB de RAM, 15 GB de disco
SSH y SFTP	OpenSSH 10.2p1
Firewall	UFW 0.36.2
Servidor web	Nginx 1.28.3, PHP-FPM 8.5.4
CMS	WordPress 7.1.4, WP-CLI 2.12.0
Base de datos	MySQL 8.4.11 LTS
Certificados	certbot 4.0.0
Cliente	Windows 11, FileZilla 3.71.1, Microsoft Edge

Los paquetes de las VM vienen del repositorio oficial de Ubuntu 26.04, que mantiene actualizaciones de seguridad durante todo el soporte de la versión. MySQL 8.4 es la versión LTS que trae la distribución. Usamos Nginx con PHP-FPM en lugar de Apache porque consume menos memoria, algo que pesa con 2 GB por VM.

```
lfmendoza@LFMENDOZA-YOGA: ~/code/school/Lab5-Redes
lfmendoza@LFMENDOZA-YOGA:~/code/school/Lab5-Redes$ qemu-system-x86_64 --version | head -1
QEMU emulator version 8.2.2 (Debian 1:8.2.2+ds-0ubuntu1.18)
lfmendoza@LFMENDOZA-YOGA:~/code/school/Lab5-Redes$ ls -l /dev/kvm
crw-rw---- 1 root kvm 10, 232 Nov  9 21:36 /dev/kvm
lfmendoza@LFMENDOZA-YOGA:~/code/school/Lab5-Redes$ pgrep -a qemu-system | cut -c1-100
35911 qemu-system-x86_64 -name lab5-web -machine q35,accel=kvm -cpu host -smp 2 -m 2048 -drive if=vi
35943 qemu-system-x86_64 -name lab5-db -machine q35,accel=kvm -cpu host -smp 2 -m 2048 -drive if=vir
lfmendoza@LFMENDOZA-YOGA:~/code/school/Lab5-Redes$ ip -br addr show br-lab5
br-lab5          UP             192.168.56.1/24
lfmendoza@LFMENDOZA-YOGA:~/code/school/Lab5-Redes$ ip -br link show master br-lab5
tap-web          UP             92:78:82:3f:bd:95 <BROADCAST,MULTICAST,UP,LOWER_UP>
tap-db           UP             b2:1d:a3:fd:76:f8 <BROADCAST,MULTICAST,UP,LOWER_UP>
lfmendoza@LFMENDOZA-YOGA:~/code/school/Lab5-Redes$
```

Figura 1. Anfitrión: QEMU 8.2.2, /dev/kvm, las dos VM en ejecución y el puente br-lab5.

3. Topología y direccionamiento

Cada VM tiene dos interfaces. `enp0s8` está en la red del laboratorio, 192.168.56.0/24, que en el anfitrión es el puente `br-lab5`; ahí se ofrecen todos los servicios y las direcciones son fijas. `enp0s3` usa la NAT de QEMU y solo sirve para que la VM salga a Internet (paquetes, Let's Encrypt y la API de DuckDNS). Desde fuera no se puede iniciar ninguna conexión hacia las VM por esa interfaz.

El cliente Windows llega a la red del laboratorio con una ruta estática hacia el anfitrión, que reenvía el tráfico y lo traduce a la dirección 192.168.56.1. Por eso las VM ven siempre al cliente con esa dirección y las reglas del firewall no dependen de la red del cliente.

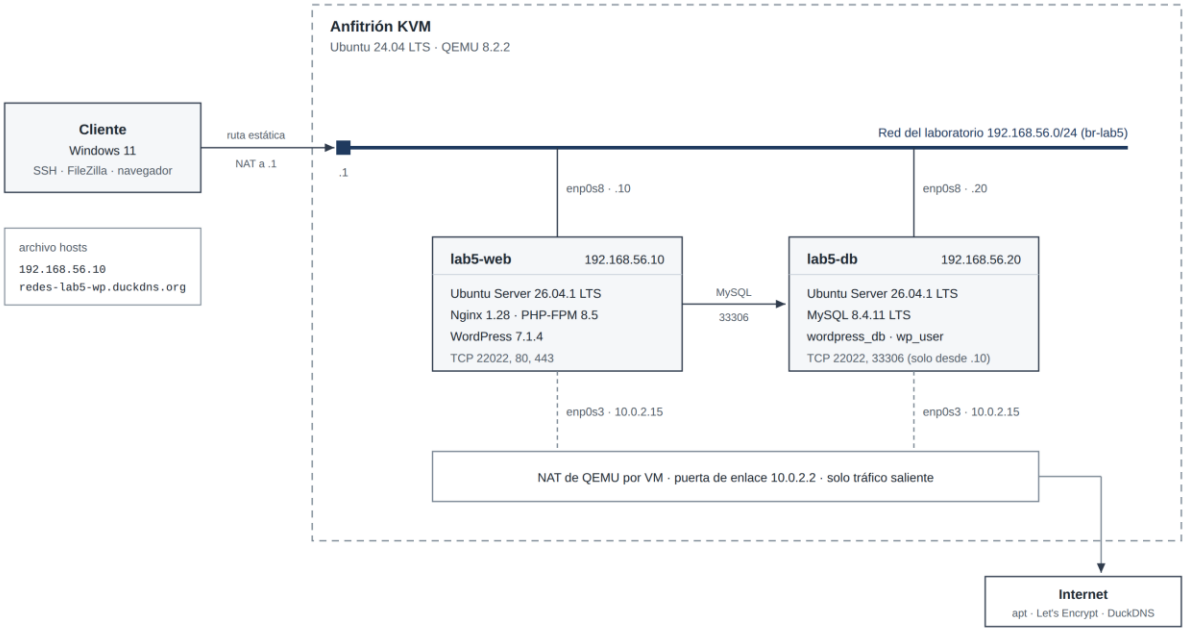


Figura 2. Topología del laboratorio.

Tabla 2. Direcciones y puertos.

Equipo	Dirección	Puertos abiertos
Anfitrión (br-lab5)	192.168.56.1	ninguno
lab5-web	192.168.56.10	22022 (SSH/SFTP), 80, 443
lab5-db	192.168.56.20	22022 (SSH), 33306 (MySQL, solo desde .10)
redes-lab5-wp.duckdns.org	192.168.56.10	resuelto por el archivo hosts

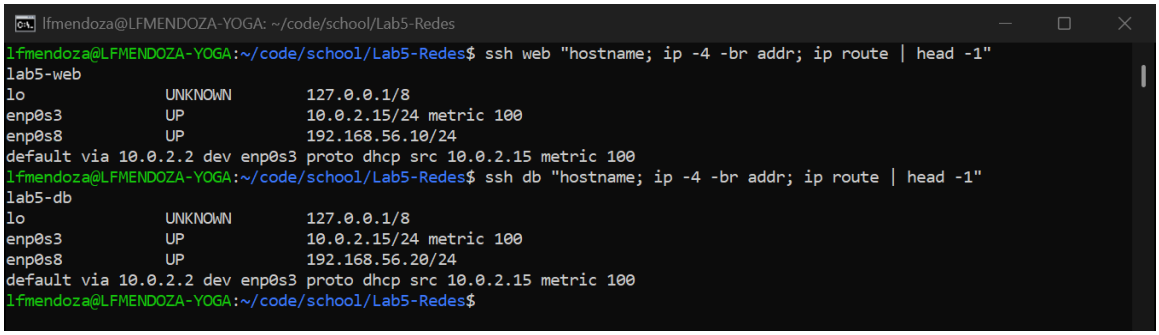


Figura 3. Interfaces de las dos VM vistas desde el anfitrión.

4. Implementación

Los scripts de las VM se ejecutan desde el cliente con `ssh web "cd ~/lab5 && sudo bash scripts/..."`. El orden importa: primero SSH y firewall, después la base de datos, porque la instalación de WordPress necesita conectarse a ella, y al final la capa web y HTTPS.

4.1 Anfitrión y máquinas virtuales

`infra-qemu/requisitos.sh` instala QEMU y las herramientas de imágenes, agrega el usuario al grupo `kvm` y crea el servicio `lab5-red`. Ese servicio levanta en cada arranque el puente `br-lab5` con 192.168.56.1/24 y las interfaces `tap-web` y `tap-db`. Las interfaces tap pertenecen al usuario, así QEMU no necesita correr como root.

`infra-qemu/crear-vms.sh` descarga la imagen de Ubuntu Server 26.04.1, crea un disco qcow2 de 15 GB para cada VM y genera los datos de cloud-init: nombre del equipo, usuario `sysadmin` con la llave pública del grupo, SSH sin contraseña y la configuración de red. Las tarjetas de red van en las ranuras PCI 3 y 8 para que Ubuntu las nombre `enp0s3` y `enp0s8`. Las VM arrancan sin pantalla (`-display none`), así que desde ese momento la única forma de entrar es por SSH.

En el cliente, `infra-qemu/windows.ps1` agrega la ruta a 192.168.56.0/24, la línea del dominio en el archivo hosts y los alias `ssh web` y `ssh db`.

4.2 Acceso por SSH

`scripts/comun/01-ssh-puerto.sh` mueve SSH al puerto 22022 y deshabilita el acceso como root y la autenticación por contraseña. La configuración va en `/etc/ssh/sshd_config.d/10-lab5.conf` y se

valida con `sshd -t` antes de reiniciar, para no perder el acceso. En Ubuntu 26.04 el puerto lo abre `ssh.socket`, así que el script también recarga systemd y reinicia el socket.

El cambio de puerto no impide el acceso a nadie; eso lo hacen la llave y el firewall. Lo que sí reduce son los intentos automáticos que buscan el puerto 22.

```
PS C:\Users\lfmen> ssh web hostname
lab5-web
PS C:\Users\lfmen> ssh db hostname
lab5-db
PS C:\Users\lfmen> ssh -p 22 -o ConnectTimeout=5 sysadmin@192.168.56.10
ssh: connect to host 192.168.56.10 port 22: Connection timed out
PS C:\Users\lfmen> ssh -p 22 -o ConnectTimeout=5 sysadmin@192.168.56.20
ssh: connect to host 192.168.56.20 port 22: Connection timed out
PS C:\Users\lfmen>
```

Figura 4. SSH por el puerto 22022. El puerto 22 ya no responde.

4.3 Firewall

`scripts/comun/02-ufw.sh` deja UFW con la política de denegar todo lo entrante y abre solo lo necesario. En lab5-web: 22022 desde la red del laboratorio, 80 (que solo redirige a HTTPS) y 443. En lab5-db: 22022 desde la red del laboratorio y 33306 únicamente desde 192.168.56.10, de modo que el cliente no puede conectarse a MySQL aunque esté en la misma red.

```
PS C:\Users\lfmen> ssh web "sudo ufw status verbose"
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22022/tcp                  ALLOW IN    192.168.56.0/24           # SSH/SFTP lab5
80/tcp                     ALLOW IN    Anywhere                  # HTTP (redirige a HTTPS)
443/tcp                    ALLOW IN    Anywhere                  # HTTPS WordPress
80/tcp (v6)                ALLOW IN    Anywhere (v6)             # HTTP (redirige a HTTPS)
443/tcp (v6)               ALLOW IN    Anywhere (v6)             # HTTPS WordPress

PS C:\Users\lfmen> ssh db "sudo ufw status verbose"
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22022/tcp                  ALLOW IN    192.168.56.0/24           # SSH/SFTP lab5
33306/tcp                  ALLOW IN    192.168.56.10             # MySQL desde web

PS C:\Users\lfmen>
```

Figura 5. Reglas de UFW en lab5-web y lab5-db.

4.4 Base de datos

`scripts/bd/01-mysql.sh` instala MySQL 8.4 y agrega `/etc/mysql/mysql.conf.d/zz-lab5.cnf` con `bind-address = 192.168.56.20`, `port = 33306` y `mysqlx = 0`. El nombre empieza con zz porque MySQL lee esos archivos en orden alfabético y se queda con el último valor; con otro nombre, `mysqld.cnf` volvía a dejar el servicio en 127.0.0.1.

La base `wordpress_db` y el usuario `wp_user` son exclusivos de WordPress. `wp_user` solo puede conectarse desde 192.168.56.10 y solo tiene privilegios sobre esa base. El usuario root de MySQL usa `auth_socket`: únicamente entra con `sudo mysql` desde la propia VM, que es como administramos la base por SSH.

```
PS C:\Users\lfmen> ssh db "sudo mysql -e 'SHOW DATABASES; SELECT user,host,plugin FROM mysql.user;'"
Database
information_schema
mysql
performance_schema
sys
wordpress_db
user    host    plugin
wp_user 192.168.56.10   caching_sha2_password
debian-sys-maint        localhost       auth_socket
mysql.infoschema        localhost       caching_sha2_password
mysql.session   localhost       caching_sha2_password
mysql.sys       localhost       caching_sha2_password
root    localhost       auth_socket
PS C:\Users\lfmen> ssh db "sudo ss -tlnp | grep mysqld"
LISTEN 0      151    192.168.56.20:33306      0.0.0.0:*    users:(("mysqld",pid=3178,fd=18))
PS C:\Users\lfmen> ssh db "mysql --version"
mysql  Ver 8.4.11-0ubuntu0.26.04.1 for Linux on x86_64 ((Ubuntu))
PS C:\Users\lfmen>
```

Figura 6. Administración de MySQL por SSH: bases, usuarios y puerto.

4.5 Servidor web y WordPress

`scripts/web/01-stack-web.sh` instala Nginx, PHP-FPM con las extensiones de WordPress, el cliente `mysql` y WP-CLI. `scripts/web/02-wordpress-permisos.sh` primero prueba la conexión a la base remota con `wp_user` y luego instala WordPress con WP-CLI, con la base en 192.168.56.20:33306. En `wp-config.php` activamos `DISALLOW_FILE_MODS` y `DISALLOW_FILE_EDIT`, porque el servidor web no tiene permiso de escritura (sección 4.6), y `FORCE_SSL_ADMIN` para que el panel solo funcione por HTTPS.

```
PS C:\Users\lfmen> ssh web "ping -c 3 192.168.56.20"
PING 192.168.56.20 (192.168.56.20) 56(84) bytes of data.
64 bytes from 192.168.56.20: icmp_seq=1 ttl=64 time=1.36 ms
64 bytes from 192.168.56.20: icmp_seq=2 ttl=64 time=4.57 ms
64 bytes from 192.168.56.20: icmp_seq=3 ttl=64 time=2.22 ms

--- 192.168.56.20 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 1.360/2.716/4.573/1.358 ms
PS C:\Users\lfmen> ssh web "nc -zv -w3 192.168.56.20 33306"
Connection to 192.168.56.20 33306 port [tcp/*] succeeded!
PS C:\Users\lfmen> ssh web "nc -zv -w3 192.168.56.20 3306"
nc: connect to 192.168.56.20 port 3306 (tcp) timed out: Operation now in progress
PS C:\Users\lfmen> ssh web "cd ~/lab5 && . config/lab5.env && . secretos.env && mysql -h $DB_IP -P $MYSQL_PORT -u $DB_US
ER -p$DB_PASS -e 'SELECT CURRENT_USER(), @@hostname, @@port; SHOW TABLES FROM wordpress_db;'"
mysql: [Warning] Using a password on the command line interface can be insecure.
CURRENT_USER()  @@hostname      @@port
wp_user@192.168.56.10   lab5-db 33306
Tables_in_wordpress_db
wp_commentmeta
wp_comments
wp_links
wp_options
wp_postmeta
wp_posts
wp_term_relationships
wp_term_taxonomy
wp_termmeta
wp_terms
wp_usermeta
wp_users
PS C:\Users\lfmen>
```

Figura 7. Desde lab5-web: ping a la base, puertos 33306 y 3306, y login de wp_user.

4.6 Permisos

El sitio pertenece a `wpadmin`, el único usuario que puede escribir en él. El grupo es `www-data`, con el que corren Nginx y PHP-FPM, que así pueden leer los archivos pero no modificarlos. Los demás usuarios, incluido `sysadmin`, no tienen acceso. Si alguien aprovechara una falla de WordPress, su código correría como `www-data` y no podría cambiar el sitio.

Tabla 3. Permisos del sitio.

Ruta	Dueño:grupo	Modo
/var/www	root:root	755
/var/www/wordpress y subdirectorios	wpadmin:www-data	2750
Archivos del sitio	wpadmin:www-data	640

El bit setgid (el 2 de 2750) hace que lo que sube `wpadmin` herede el grupo `www-data`. `/var/www` pertenece a root porque así lo exige el chroot de SFTP.

```
chown -R wpadmin:www-data /var/www/wordpress
find /var/www/wordpress -type d -exec chmod 2750 {} +
find /var/www/wordpress -type f -exec chmod 640 {} +
```

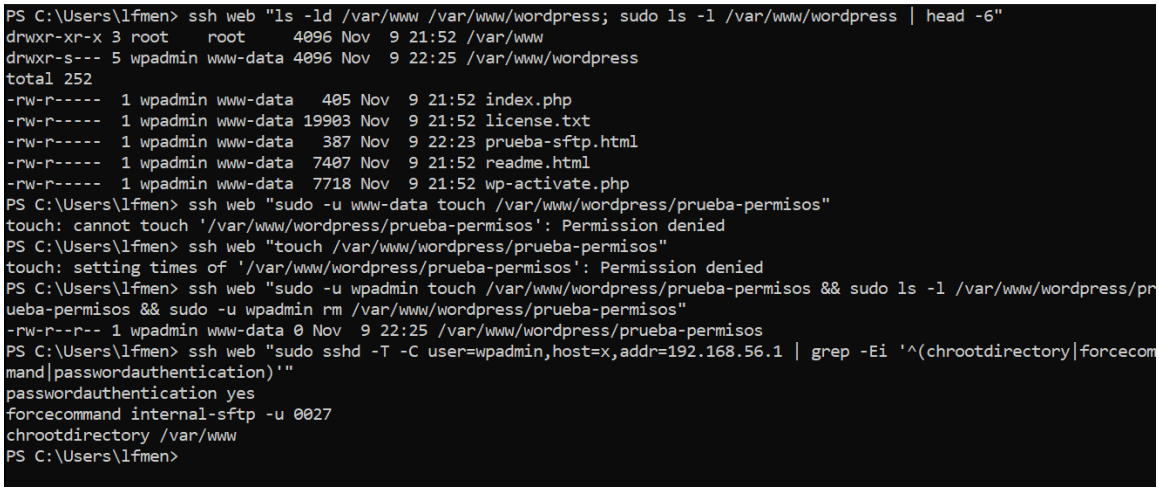


Figura 8. Permisos resultantes y prueba de escritura con www-data, sysadmin y wpadmin.

4.7 SFTP y FileZilla

`scripts/web/03-sftp.sh` agrega a sshd un bloque `Match User wpadmin` con `ChrootDirectory /var/www`, `ForceCommand internal-sftp -u 0027` y autenticación por contraseña solo para ese usuario. `wpadmin` no puede abrir una terminal, solo ve la carpeta `wordpress` y lo que sube queda con permisos 640 o 750.

En FileZilla nos conectamos a `sftp://192.168.56.10` en el puerto 22022 como `wpadmin`. En la primera conexión comparamos la huella de la llave que mostró FileZilla con la de lab5-web (`ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub`) antes de aceptarla.

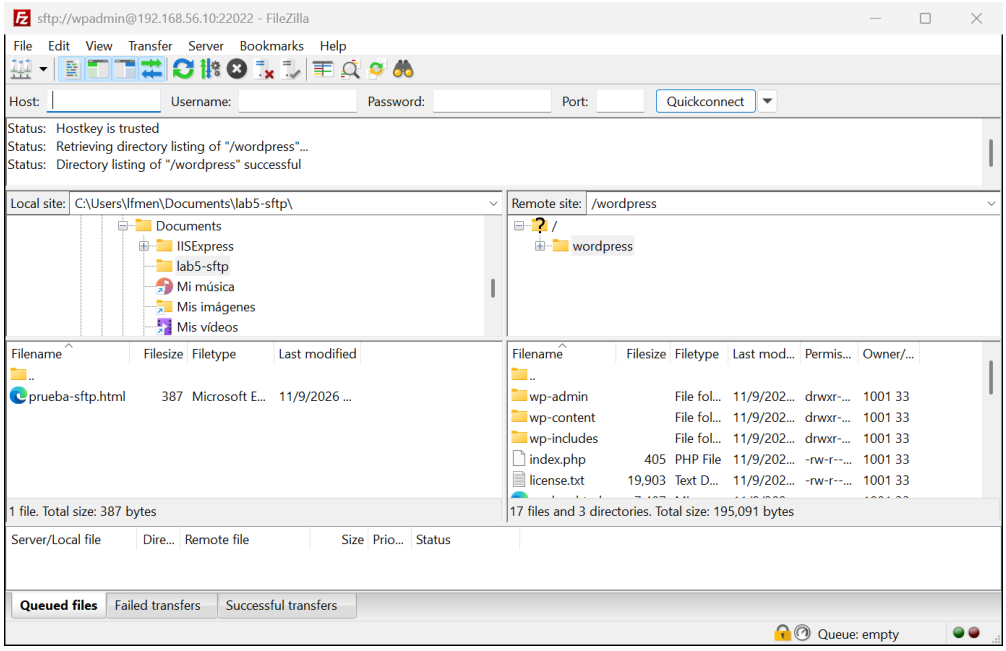


Figura 9. FileZilla conectado como wpadmin. Solo se ve /wordpress.

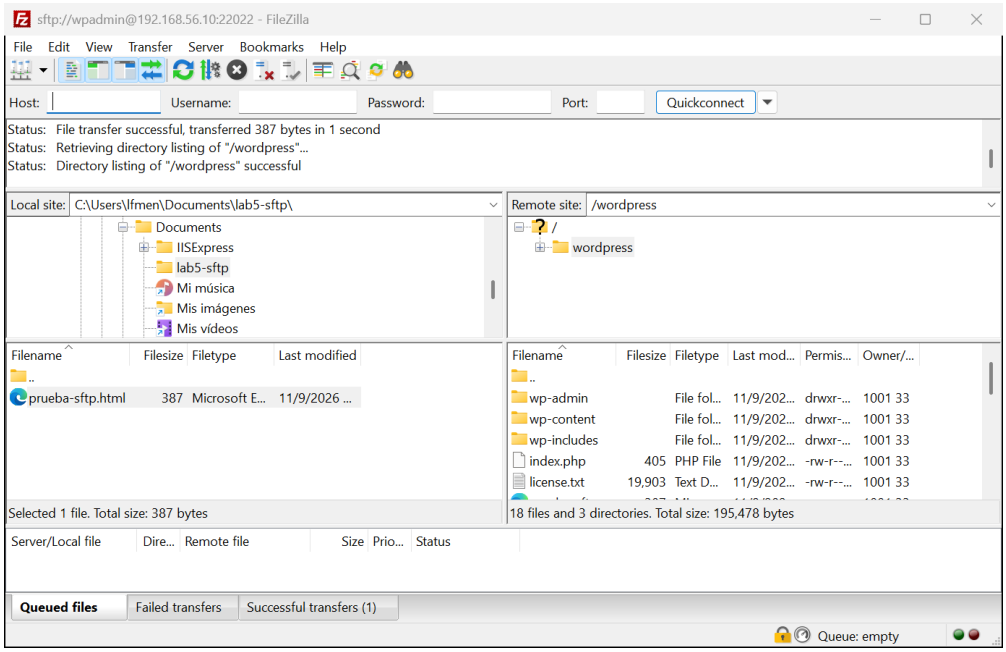


Figura 10. Subida de prueba-sftp.html.



Figura 11. El archivo subido, servido por HTTPS.

4.8 HTTPS

Let's Encrypt solo emite un certificado si se demuestra control sobre el dominio. Los retos HTTP-01 y TLS-ALPN-01 requieren que sus servidores se conecten a la VM por el puerto 80 o 443, y nuestras VM no son accesibles desde Internet. Usamos el reto DNS-01, que solo pide publicar un registro TXT en el DNS público. Por eso el dominio es un subdominio de DuckDNS, que permite crear ese registro por API; un dominio inventado como `.local` no se puede validar.

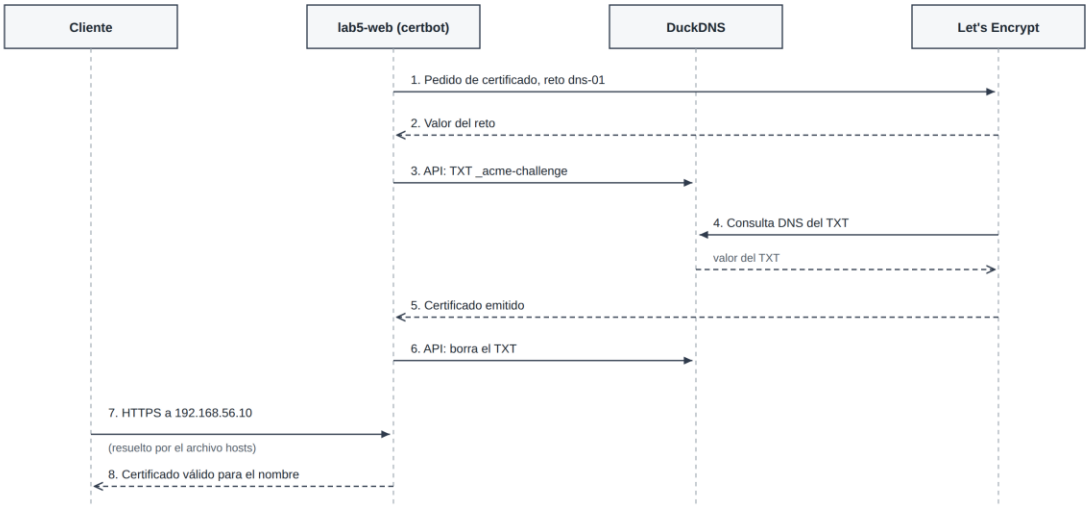


Figura 12. Emisión del certificado con el reto DNS-01.

`scripts/web/04-https.sh` instala certbot con dos scripts que publican y borran el TXT en DuckDNS. Antes de pedir el certificado real lo probamos contra el entorno de pruebas de Let's Encrypt, que no

cuenta para el límite de emisiones. Nginx redirige el puerto 80 a HTTPS, acepta TLS 1.2 y 1.3, envía HSTS y bloquea `wp-config.php` y `xmlrpc.php`. `certbot.timer` se encarga de la renovación.

El nombre existe en el DNS público solo para la validación. En el cliente, el archivo hosts lo dirige a 192.168.56.10, y el navegador acepta el certificado porque coinciden el nombre y la cadena de confianza; la dirección IP no interviene.

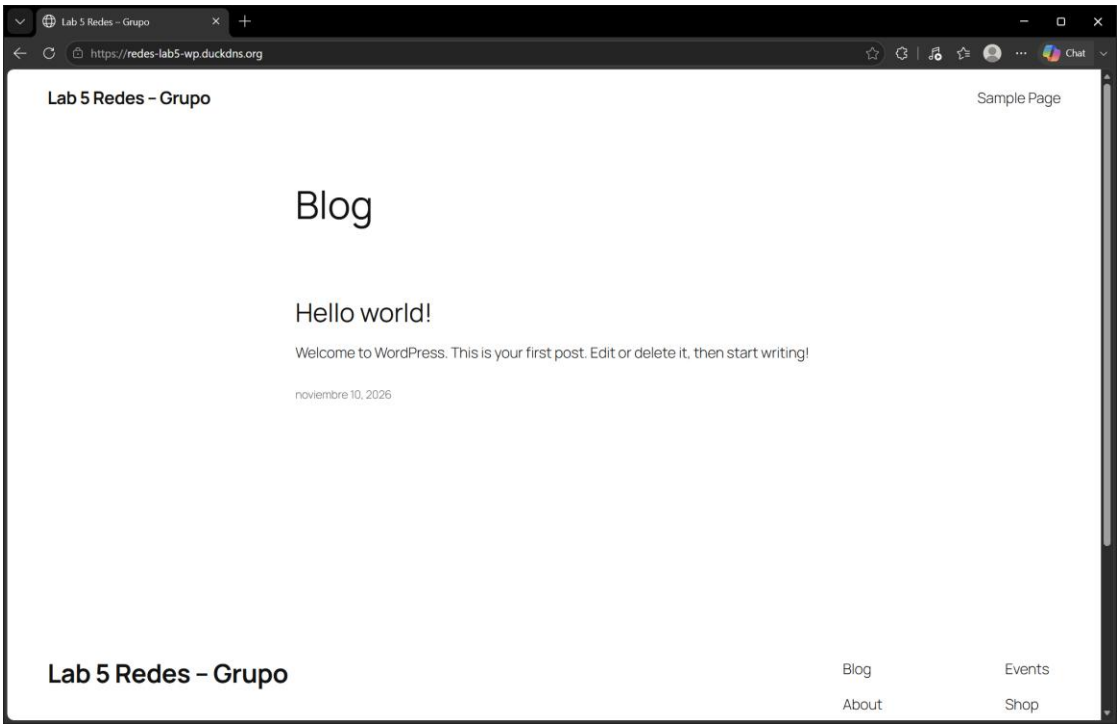


Figura 13. WordPress por HTTPS en el navegador del cliente.

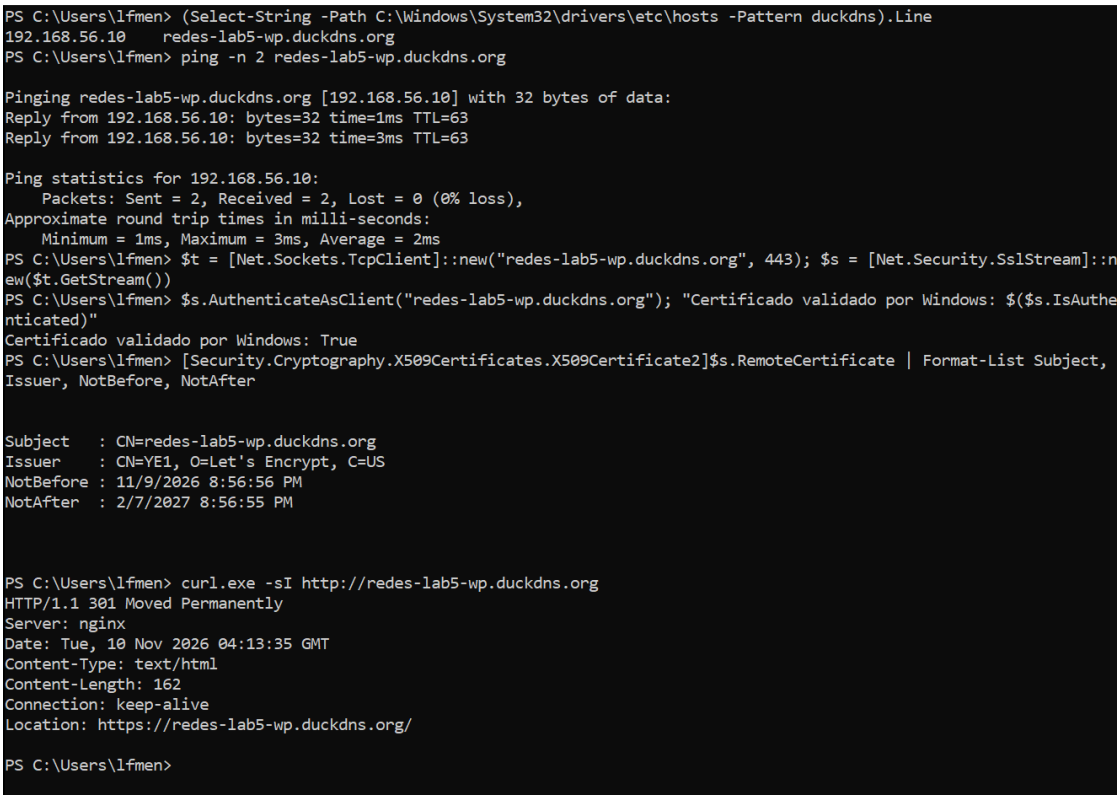


Figura 14. Archivo hosts, ping al dominio, validación del certificado en Windows y redirección de HTTP.

4.9 Resolución local

El archivo `C:\Windows\System32\drivers\etc\hosts` del cliente tiene la línea `192.168.56.10 redes-lab5-wp.duckdns.org`. Windows consulta ese archivo antes que el DNS, así que el nombre resuelve a la VM web aunque en DuckDNS apunte a otra dirección (figura 14).

5. Cliente y servidor

Una misma máquina puede ser servidor en una conexión y cliente en otra. lab5-web es el servidor que atiende al navegador, pero frente a la base de datos es el cliente: PHP y el cliente `mysql` se conectan a `mysqld` en lab5-db.

Tabla 4. Clientes y servidores en cada conexión.

Conexión	Cliente (aplicación, equipo)	Servidor (aplicación, equipo)
Sitio web	Edge, Windows	Nginx y PHP-FPM, lab5-web
Archivos	FileZilla, Windows	OpenSSH (SFTP), lab5-web
Administración	ssh, Windows	sshd, lab5-web y lab5-db
Datos	PHP y mysql, lab5-web	mysqld, lab5-db

6. Pruebas

Las pruebas están en `scripts/pruebas`: `pruebas-servidor.sh` corre dentro de cada VM y `pruebas-host-windows.ps1` desde el cliente. Las de telnet las hicimos a mano.

Tabla 5. Resultados de las pruebas.

Prueba	Resultado
ping entre lab5-web y lab5-db	0 % de pérdida
lab5-web hacia 192.168.56.20:33306	conecta (nc; telnet muestra el saludo de MySQL 8.4.11)
lab5-web hacia 192.168.56.20:3306	sin respuesta
Login de wp_user desde lab5-web	wp_user@192.168.56.10, 12 tablas de WordPress
Cliente hacia 22022 y 443	abiertos
Cliente hacia 22 y 33306	sin respuesta
Escritura en el sitio	solo wpadmin; www-data y sysadmin reciben Permission denied
SFTP con FileZilla	archivo subido y visible por HTTPS
HTTPS	200, certificado de Let's Encrypt válido
HTTP	301 hacia HTTPS



Figura 15. telnet desde lab5-web al puerto 33306.

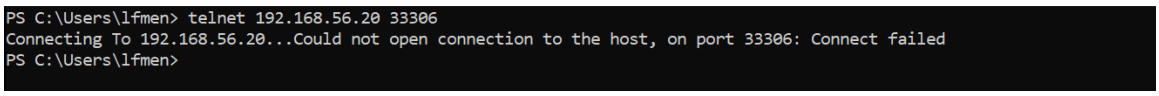


Figura 16. telnet desde el cliente al puerto 33306.



Figura 17. telnet desde el cliente al puerto 443: la conexión queda abierta.

```
PS C:\Users\lfmen> (Test-Connection 192.168.56.10, 192.168.56.20 -Count 1 | Format-Table Address, IPV4Address, ResponseT
ime -AutoSize | Out-String).Trim()
Address       IPV4Address   ResponseTime
-------       -----------   ------------
192.168.56.10 192.168.56.10           1
192.168.56.20                         3
PS C:\Users\lfmen> (Resolve-DnsName redes-lab5-wp.duckdns.org -Type A | Format-Table Name, IPAddress -AutoSize | Out-Str
ing).Trim()
Name                    IPAddress
----                    ---------
redes-lab5-wp.duckdns.org 192.168.56.10
PS C:\Users\lfmen> "192.168.56.10:22022","192.168.56.10:443","192.168.56.20:22022","192.168.56.10:22","192.168.56.20:22"
,"192.168.56.20:33306" | ForEach-Object { $h,$p = $_ -split ":"; Test-NetConnection $h -Port $p -WarningAction SilentlyC
ontinue } | Format-Table ComputerName, RemotePort, TcpTestSucceeded -AutoSize

ComputerName  RemotePort TcpTestSucceeded
------------  ---------- ----------------
192.168.56.10      22022             True
192.168.56.10        443             True
192.168.56.20      22022             True
192.168.56.10         22            False
192.168.56.20         22            False
192.168.56.20      33306            False


PS C:\Users\lfmen> ssh web "hostname; lsb_release -d"
lab5-web
Description:    Ubuntu 26.04.1 LTS
PS C:\Users\lfmen> ssh db "hostname; lsb_release -d"
lab5-db
Description:    Ubuntu 26.04.1 LTS
PS C:\Users\lfmen> curl.exe -sI https://redes-lab5-wp.duckdns.org
HTTP/1.1 200 OK
Server: nginx
Date: Tue, 10 Nov 2026 04:18:32 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
Link: <https://redes-lab5-wp.duckdns.org/index.php?rest_route=/>; rel="https://api.w.org/"
Strict-Transport-Security: max-age=31536000

PS C:\Users\lfmen>
```

Figura 18. Pruebas desde el cliente Windows.

7. Problemas encontrados

Tabla 6. Problemas y soluciones.

Problema	Solución
En el primer arranque la interfaz NAT apareció como enp0s2; cloud-init no puede renombrar una interfaz activa.	Fijamos las tarjetas en las ranuras PCI 3 y 8.
El script entraba por SSH antes de que terminara el arranque y el login era rechazado.	Esperar a que el login funcione, no solo a que el puerto abra.
MySQL escuchaba en 127.0.0.1.	Renombrar la configuración a zz-lab5.cnf.
El puerto de SSH no cambiaba al reiniciar el servicio.	Reiniciar ssh.socket.
El cliente llegaba a las VM con una IP fuera de 192.168.56.0/24 y UFW bloqueaba SSH.	Traducir el origen a 192.168.56.1 en el anfitrión.
FileZilla no está disponible en winget.	Usamos el zip oficial y verificamos su SHA-512.

8. Conclusiones

Separar el servidor web y la base de datos solo sirve si la red lo respeta. Con `bind-address`, la regla de UFW y un usuario limitado a su base, MySQL no es accesible ni desde el cliente.

Con los permisos queda claro quién hace qué: `wpadmin` publica, `www-data` sirve y nadie más toca los archivos.

El reto DNS-01 nos permitió tener un certificado público para un servidor privado. Y como todo está en scripts, reconstruir el laboratorio completo toma unos minutos.

Referencias

[1] QEMU Project. QEMU System Emulation User's Guide. https://www.qemu.org/docs/master/

[2] Linux Kernel Documentation. KVM. https://docs.kernel.org/virt/kvm/

[3] Canonical. cloud-init: NoCloud y Networking config v2. https://docs.cloud-init.io/

[4] Canonical. Ubuntu Server documentation: OpenSSH, UFW, Netplan. https://documentation.ubuntu.com/server/

[5] Oracle. MySQL 8.4 Reference Manual: Using Option Files. https://dev.mysql.com/doc/refman/8.4/en/

[6] OpenBSD. sshd_config(5). https://man.openbsd.org/sshd_config

[7] NGINX. ngx_http_ssl_module. https://nginx.org/en/docs/

[8] Let's Encrypt. Challenge Types; Rate Limits. https://letsencrypt.org/docs/

[9] EFF. Certbot User Guide. https://eff-certbot.readthedocs.io/

[10] DuckDNS. Spec. https://www.duckdns.org/spec.jsp

[11] WordPress.org. Hardening WordPress; WP-CLI. https://developer.wordpress.org/

Anexo. Reconstrucción del laboratorio

```
# En el anfitrión (una sola vez)
sudo bash infra-qemu/requisitos.sh

# Crear y arrancar las VM
infra-qemu/crear-vms.sh

# Desplegar todo y guardar las salidas en evidencias/qemu/
infra-qemu/desplegar.sh --certificado

# En el cliente Windows, PowerShell como administrador
powershell -ExecutionPolicy Bypass -File infra-qemu\windows.ps1
```